CCE - II

UNIT-3

Page No:
Date.

* change suggested in the updated project.
- The previous project was a basic circular linked list program that could only add bus stops & display the Route. The updated project include more advanced data strcure and additional operations.
such as.
- circular & double linked list
- Add & deleting bus stops.
- Forward & Backward Root display.
- searching stops
- Display Next & previous stop.
- stack & queue Based operations.

* New problem statement.
The previous Bus Root management system used a circular linked list to store bus stop & display the route in sequence. However, it lacks essential features. such as deleting stops, displaying the route in reverse order, and managing pending route updates.
To oversome These limitations, the, system is enhanced using a circular doubly linked-list., stack & queue. The circular doubly linked list allows bus stops to be traversed in both Forward & Backward direction. The stack is used to undo the most recent Addition. While the queue Manages pending Bus stops according to FIFO principle.

Page No:
Date.

* solution :-
The proposed solution is a menue driven C++ program that combines three data strucures to improve bus route management.

1. circular doubly linked list :- store bus stop using next & previous pointers. it supports insertion, deletion, & searching / forward traversal & Backward traversal.

2. stack :- stores the name of newly Added stop so the most recent Addition can be undone using the LIFO principle.

3. Queue :- stores pending bus stops. user can add stop to the queue, remove the first pending stop, or process a pending stop and add it to the main route.

* objectives :-
- To implement a circular doubly linked list for bus route management.
- To insert & delete bus stop dynamically.
- To search for an particular Bus stop & display it's position
- To display the Bus route in Forward & Backward direction.
- To identify next & previous stop for a given stop.
- To implement a stack of undoing the last recorded addition.

- To implement a queue for managing pending Bus stops.
- To understand the practical application of linked list, stack & queue.. in C#+.
- To improve the flexibility and efficiency of bus route operations.

* Algorithm:

Step 1: Start.

Step 2: Initialize the circular doubly linked list, stack, and queue.

Step 3: Insert four default Bus stops: Sangamneer, Akole, Shirdi, and Nasik.

Step 4: Display the main menue & accept the users choice.

Step 5: if choice is 1, add new bus stop and push it's name on to the stack.

Step 6: if the choice is 2, search for specified Bus stop and. delete if it found.

Step 7: if the choice is 3, search for Bus stop & display its possition.

Step 8: if the choice is 4, display the Bus route in forward direction.

Step 9: if the choice is 5, display the current, previous & Next stop.

Step 10: if the choice is 6, display the current, previous & Next stope.

Step 11: if the choice is 7, undo the last recorded addition using stack.

Page No:
Date.

Step 12: If the choice is 8, open Queue management to add delete, or ~~pos~~ process pending stops.

Step 13: If the choice is 9, free the remaining linked-list nodes & exit the program.

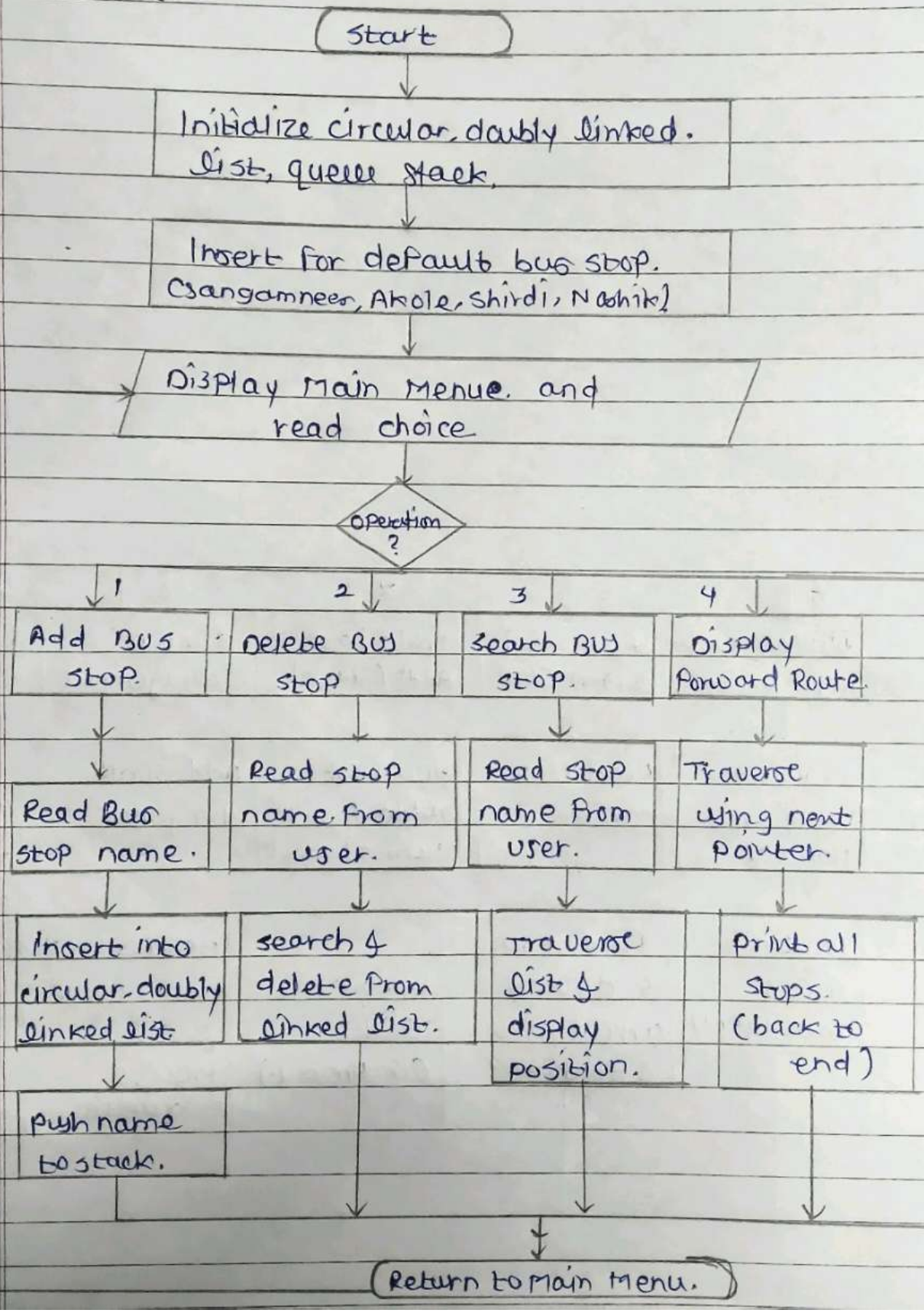
Step 14: If the choice is invalid, display an error message.

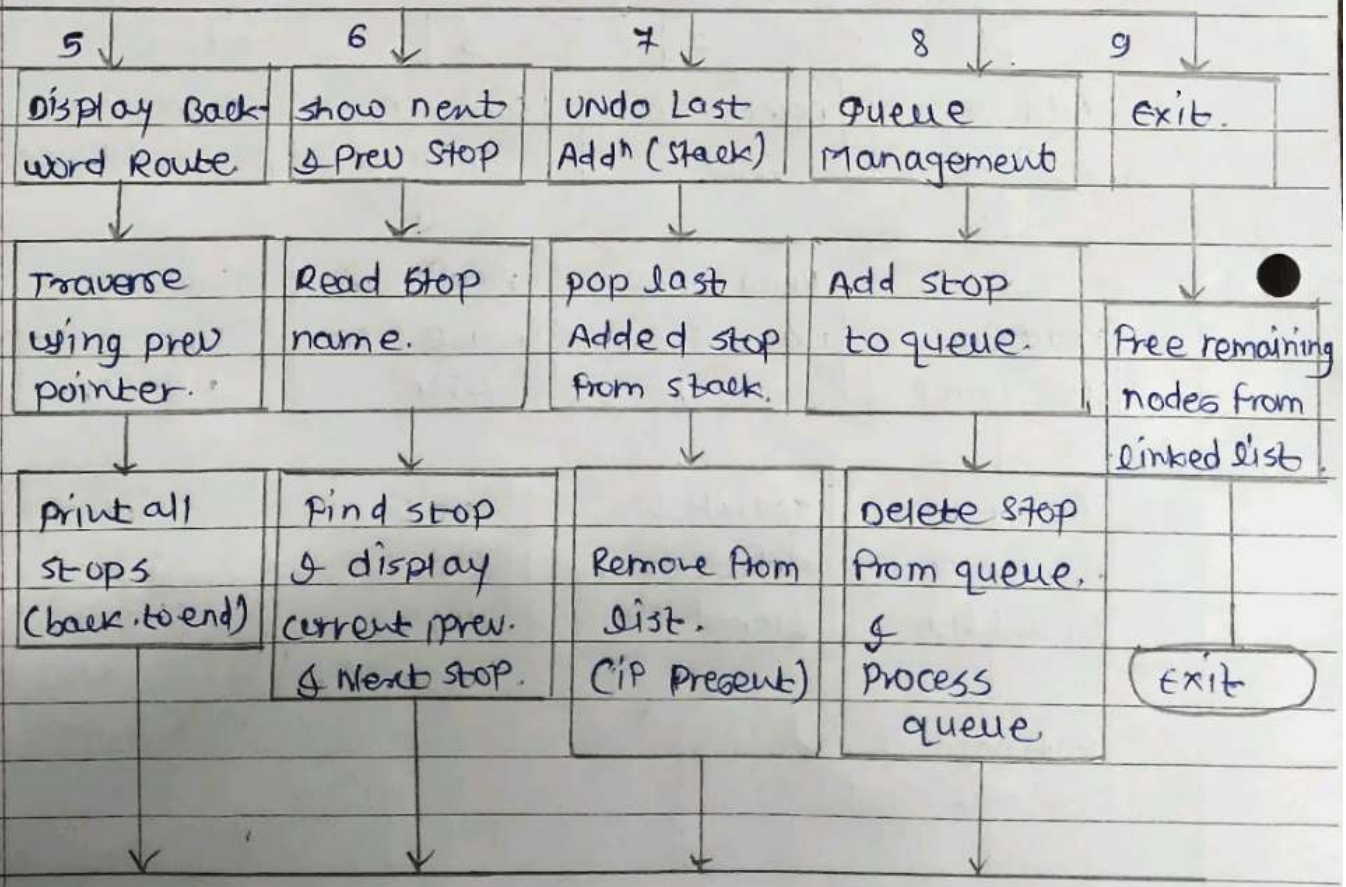
Step 15: Repeat the menue until user choose exit.

Step 16: Exit.

Page No:
Date.

* Flowchart.




5
6
7
8
9
Display Back-ward Route
show next & prev Stop
UNdo Last Add^n (Stack)
Queue Management
Exit.
Traverse using prev pointer.
Read Stop name.
pop last Added stop from stack.
Add stop to queue.
Free remaining nodes from linked list
print all stops (back to end)
Find stop & display current prev. & Next stop.
Remove from list. (if present)
Delete stop from queue. & Process queue
Exit